

Input Order Is an Uncontrolled Variable in Lossless Point Cloud Compression Benchmarks

September 20, 2026

Abstract

Lossless point cloud codecs are compared at equal bitrate, but they do not all encode the same object. Some reconstruct the multiset of points; others reconstruct the sequence. The arrangement of the input is never part of the test conditions, so a bitrate reported without it is underdetermined. We make the arrangement an explicit factor: eight order conditions applied to the same point set, four lossless coders, 26 blocks drawn from 12 files, 204 rows, each covering four coders.

The effect is large and coder-dependent. Under a uniform random permutation the geometry code length of LASzip rises by a median of 121.9% and that of a sequential context coder by 130.9%, while a single-slice G-PCC configuration agrees with the identity order to three significant digits in all 204 rows. Two null controls are needed, not one: reversing the input is a structured treatment whose spread (-1.28% to $+1.59\%$ for one coder) is five times the seed-to-seed noise, and using the smaller control alone inflates the count of blocks that appear to respond from 6 to 15.

A coder that derives its coding order from a sort key loses that protection as the key degenerates, but not as a clean dichotomy. Against our own effective floor, blocks whose key duplication is at most 45% are indistinguishable from a benign rearrangement with one exception; a file at 82.4% sits 4.4 times the floor; fully degenerate keys sit 300 to 500 times it. A controlled sweep that manipulates the key alone, on two files, places the excess over the benign control at full degeneracy in one file and spreads it across intermediate levels in the other. We therefore withdraw our earlier claim that injectivity of the key is the governing condition — our own data refute it — without replacing it with a sharp threshold. We conclude with the minimum an order specification must fix for lossless comparisons to mean anything.

1 Introduction

A lossless point cloud codec is judged by bits per point. The comparison assumes that the coders under test are solving the same problem. They are not.

An octree coder such as G-PCC reconstructs the *multiset* of occupied positions. The order in which the points arrived is discarded, and the decoder is free to emit them in whatever order the traversal produces. A sequential coder such as LASzip reconstructs the *sequence*: point i of the output is point i of the input. For $n = 10^6$ the difference is $\log_2(n!)/n = 18.489$ bits per point of information that one coder carries and the other does not. That is comparable to the entire geometry budget of an airborne LiDAR tile.

This is known in the abstract. What is missing is a measurement. The common test conditions used to compare lossless point cloud coders do not specify the arrangement of the input, and no published study varies it systematically. A bitrate reported without an order condition is therefore not reproducible: the same coder on the same points can differ by more than a factor of two depending on how the file happened to be written.

We treat the arrangement as an experimental factor. Section 3 defines eight order conditions and, importantly, two null controls. The second control is the part that is easy to get wrong, and we got it wrong first: reversing the input is not noise, it is a treatment, and its spread sets the resolution of the whole experiment.

1.1 Contributions

The gap in the literature is narrow and we state it narrowly: the measurement itself. Every component of the invariance property is prior art (Section 2).

1. A protocol that makes input order an explicit factor, with two null controls and a stated resolution, and the measurement it produces: four coders, 26 blocks, eight conditions, 204 rows (Sections 3 and 5).
2. A withdrawal. We claimed that injectivity of the sort key governs the order invariance of a key-sorted coder. Our own data refute it, and we report what does and does not take its place (Section 6).

2 Related work

Order-invariant geometry coding is standard. The octree of G-PCC and the kd-tree of Draco both code a multiset, so their geometry cost does not depend on the input arrangement. TMC13 additionally offers `--sortInputByAzimuth` and `--predGeomSort`, which re-derive an acquisition-like order internally, and a `canonical_point_order_flag`. Re-deriving a scan order from the data is therefore prior art, not a contribution.

Sending the order as a separate channel is patented. US10796457B2 (Van Beek, Intel, priority

Table 1: Lossless coders divide on two independent axes. The comparison at equal bitrate is only meaningful within a row.

	Cost depends on order	Cost independent
Multiset	—	G-PCC (single slice)
Sequence	LASzip, ours (v3)	ours (scan), cond.

2018) states that if point positions are coded losslessly the reconstructed X, Y, Z are identical whatever the coding order, and that the choice of coding order may be transmitted so that the original order can be reconstructed. We have not analysed the independent claims and make no statement about claim scope; the specification, at least, discloses the combination we measure. Any differentiation would have to come from coding an arbitrary permutation rather than selecting from a pre-defined set.

The $\log(n!)$ framework is established in the **opposite direction**. Shuffle coding [2] and Random Order Coding [3] spend effort to *save* the $\log(n!)$ bits of an unordered set. We are measuring what it costs to *keep* them. The machinery is the same with the sign reversed, and neither is novel here.

Re-deriving a scan order from the data is also the subject of recent learned work [5].

LASzip [1] is the representative order-dependent coder: it does not re-sort, and its predictor follows the scan line. Re-sorting is left to external tools, which destroy the input order.

What we could not find in the literature is the measurement itself: a systematic comparison of code length under controlled input orders, with a null control, for lossless point cloud coders. Our own earlier work [4] reports a single instance of the effect (one file, 2M points, one direction) and uses it to argue that reordering is not a levelling operation. It also already claims the invariance property for the *attribute* columns of the same coder, on the grounds that their coding order is derived from the geometry. The present paper measures the *geometry* columns of that coder, turns the remark into a protocol, and finds that the earlier reading was too simple: the property has a condition, and the condition is not the one we stated.

3 The protocol

3.1 Two axes, not one

Table 1 separates two properties that are usually conflated: what the decoder is required to reconstruct, and whether the code length depends on the arrangement it was given. A coder in the top row is not comparable at equal bitrate with one in the bottom row, because the bottom row is also transmitting the permutation.

3.2 Order conditions

For each block of n points we hold the point set fixed and vary only the arrangement:

- **identity** — the order in the source file (the baseline);
- **reverse** — the exact reversal;
- **Morton** — sorted by Morton code of the coordinates, each axis normalised to its own range before interleaving;
- **window** w — a uniform random permutation applied inside consecutive windows of w points, for $w \in \{10^2, 10^3, 10^4\}$, kept only when $10w \leq n$;
- **random** — a uniform random permutation of all n points, run twice with different seeds.

The window conditions interpolate between the identity and a uniform permutation, so a coder whose cost tracks local disorder should respond monotonically.

3.3 Two null controls

A measurement of this kind needs a floor below which a difference cannot be distinguished from the apparatus. We use two controls, and the distinction matters.

(a) *Seed-to-seed*. The two random conditions differ only in the seed. Their difference measures variation within a single treatment, with the baseline cancelling.

(b) *Reverse*. Reversal preserves every local neighbourhood; a sequential predictor that is symmetric in direction should be unaffected. It is nevertheless a structured rearrangement, and its deviation from zero measures what a *benign* rearrangement costs.

The effective floor is the larger of the two. This is not a formality: the two-control design came out of correcting our own first design, which used (b), and our second, which replaced it with (a). Control (b) is larger by a factor of five for one coder, and it is the one that binds.

3.4 Coders

Four lossless coders, all measured on geometry only so that the levels are comparable:

- **G-PCC** — TMC13 **release-v23.0-rc2**, lossless octree, geometry only, driven through a PLY file with **float32** coordinates, which bounds the integer range the protocol can carry;
- **LAZ** — LASzip via a point record containing only X, Y, Z . The figure is the whole file divided by n , so it includes a header and a chunk table that do not depend on the order; on the smallest block this fixed part is about 0.3 bpp, which makes the reported percentage change slightly conservative;
- **v3** — our sequential context coder over the three coordinate columns, using cross-axis context;
- **scan** — our scan-model coder, which sorts internally by the key (flight line, timestamp) before predicting.

3.5 Verification

Every row of the matrix comes from a run that was decoded and compared, and all 204 pass. For our own coder the candidate is pinned, so the configuration that is reported is the configuration that was verified, and the fallback path that would silently substitute an embedded container is disabled.

The meaning of “verified” differs by coder and we state it rather than eliding it: for LAZ and for our container it is equality of all columns *in order*; for G-PCC it is equality of the multiset of coordinates after a lexicographic sort. Two caveats. First, in the run reported here the gate accepted a G-PCC row whose decode produced no verdict as well as one that produced a positive verdict; we have since made the gate strict and re-measured the 36 rows of the five small blocks, which pass strictly with identical code lengths, but the 168 rows of the large blocks have not been re-verified under the strict gate. Second, figures quoted from other experiments — the +137.9% of Section 8 and the +27.1% of [4] — are *not* part of this matrix and were not round-trip verified per candidate; we flag them where they appear.

4 Experiments

Twelve files, all in the LAS/LAZ family: airborne LiDAR tiles (AHN3, AHN4 $\times 2$, AHN5, Autzen $\times 2$, Red Rocks, a survey block with extra bytes) and five small test files, some of them synthetic. Seven large files contribute three blocks of 10^6 points each, below the default `sliceMaxPoints` of TMC13 so that G-PCC codes a single slice; five small files are used whole (10,683 to 110,000 points). This gives 26 blocks. With eight conditions, less the four window conditions that the small blocks cannot support, $26 \times 8 - 4 = 204$ rows. The four coders are columns, not a factor.

The three blocks within a large file give, for the first time in this line of work, a spread for a quantity that was previously a single sample.

5 Results

5.1 The effect is large and coder-dependent

Table 2 is the main result. A uniform permutation of the same points costs LASzip 121.9% and our sequential coder 130.9% of their identity-order geometry budget. G-PCC agrees with the identity order to three significant digits in every one of the 204 rows, as its construction requires; at 10^6 points three digits is a resolution of 0.001 bpp, or 125 bytes, so the correct statement is “agrees to three digits”, not “exactly zero”.

The interquartile range is wide. Reporting only the median would hide it, and the spread within a single file is itself large: the three blocks of one airborne file move between +38.6% and +129.0% for the same coder and condition.

Table 2: Code length relative to the identity order, in per cent. Median over 26 blocks; the window 10^4 row is over the 22 blocks large enough to support that condition. The last row gives the interquartile range of the random condition.

Condition	G-PCC	LAZ	v3	scan
reverse	+0.0	−0.0	−0.0	−0.1
Morton	+0.0	+16.1	+23.0	−0.0
window 10^2	+0.0	+65.5	+70.7	+0.3
window 10^3	+0.0	+80.8	+86.0	+0.3
window 10^4	+0.0	+89.1	+95.6	+0.2
random 1	+0.0	+121.9	+130.9	+0.2
random 2	+0.0	+121.9	+130.9	+0.2
IQR, random	—	[89, 140]	[102, 153]	[0.0, 1.5]

Table 3: Two null controls, in percentage points. (a) seed-to-seed difference between the two random conditions; (b) deviation of the reverse condition from zero.

	(a) med.	(a) max	(b) min	(b) max	floor
G-PCC	0.000	0.000	+0.000	+0.000	0.000
LAZ	0.009	0.664	−0.385	+0.588	0.664
v3	0.015	0.261	−1.099	+0.070	1.099
scan	0.015	0.305	−1.282	+1.591	1.591

5.2 The window conditions separate two behaviours

Code length increases monotonically with the window width for LAZ (22 of 22 blocks that support all four conditions) and for v3 (22 of 22). For the scan coder it is monotone in only 3 of 22 — and those three are the three blocks of the one fully degenerate file among those 22, where the response is not noise but the largest effect in the study ($+225 \rightarrow +317 \rightarrow +413 \rightarrow +522\%$). The remaining 19 blocks are flat at the level of the floor. The other fully degenerate file is too small to support the widest window and does not enter this count.

The two order-dependent coders degrade gradually. The scan coder does not interpolate between its two behaviours: the degenerate blocks grow with the window width, while the one intermediate case saturates immediately (Section 6). A graded mechanism and a saturating one are not the same mechanism at two strengths.

5.3 Morton order is not uniformly bad

Sorting into Morton order raises the median cost by 16.1% (LAZ) and 23.0% (v3), which reproduces the direction of the single measurement in [4]. The magnitudes are not comparable with that earlier figure of 27.1%, which was one airborne file of 2M points with a different column definition.

The sign is not uniform. Morton order *reduces* the cost in 6 of 26 blocks for LAZ, 5 of 26 for v3 and 14 of 26 for the scan coder, with a range for v3 of -21.0% to $+232.3\%$. “Space-filling-curve order is bad for sequential predictors” is a statement about medians, not a rule.

Table 4: Cost of a uniform permutation, by file, against the duplication rate of the sort key. The representative block of each file is the one whose key duplication is the median; the last column gives the range over the three blocks where a file has three.

File	key dup.	LAZ	v3	scan	scan, range
red-rocks	100.0	+458	+513	+517	+510 to +522
simple1_4	100.0	+829	+642	+787	—
plane	82.4	+189	+209	+7.0	—
workshop	39.2	+133	+146	+1.8	+0.3 to +1.8
fullwave	33.7	+69	+86	+0.5	—
AHN3	24.0	+123	+134	+0.7	+0.4 to +1.3
AHN4 (b)	18.3	+136	+149	-1.1	-1.1 to +0.1
AHN5	15.5	+116	+131	+0.2	+0.1 to +1.2
autzen-trim	9.7	+119	+128	-0.4	—
AHN4 (a)	8.3	+124	+135	-0.2	-0.3 to -0.1
autzen-2023	7.3	+84	+87	+0.0	+0.0 to +0.1
vegetation	0.0	+56	+56	+0.0	—

5.4 The floor

Table 3 is the methodological point of the paper. For the scan coder the benign rearrangement moves the code length five times as far as the seed does. If the seed difference alone is taken as the floor, 15 of 26 blocks appear to respond to the order; against the effective floor, 6 do. The 9 blocks gained by the smaller floor fall inside the range that reversal itself spans — partly by construction, since the effective floor is the half-width of that range, and in fact all nine do — so they cannot be separated from a rearrangement that costs nothing.

We report this because we made the substitution ourselves, in a redesign that was otherwise an improvement, and it took an outside reading to catch it.

6 Order invariance and the sort key

The scan coder sorts internally by (flight line, timestamp) before predicting. When that key is a total order on the points, the sort undoes any input permutation and the code length cannot depend on it. Table 4 shows what happens when it is not.

The endpoints. With a fully degenerate key the coder loses the property completely: +509.7 to +787.4% over the four such blocks, and on red-rocks the scan coder (+517%) lands on the plain geometry coder (+513%) — it has collapsed into one. At the other end, the one file whose key has no duplicates is identical to three digits across all seven of its conditions. These two facts establish the dependence without any appeal to a correlation.

What we claimed and withdraw. We wrote, twice, that injectivity of the key is what decides the property. Our own data refute it. The file `plane` has an 82.4% duplicated key — far from injective — and moves by only 7.0%; one block at 45.0% moves by 0.06%. Stated as injectivity the condition is too strong: only one of the ten non-degenerate files satisfies it, and the other nine behave as if they did.

What we do not put in its place. A dichotomy

Table 5: Controlled sweep: input order fixed, key duplication manufactured by quantising the timestamp. Cost of a random permutation, per cent, against the reverse-order control at the same quantisation.

dup.	vegetation		AHN4	
	random	reverse	random	reverse
25%	-1.6	-4.6	+4.0	+4.5
50%	-0.7	-4.8	+5.9	+6.2
75%	+6.0	-2.0	+14.6	+13.9
100%	+46.2	+0.1	+125.9	-0.1
dither at 100%: +0.0 and +0.0				

between full degeneracy and everything else does not survive our own floor either. Measured against the effective floor of 1.591 percentage points (Section 5.4), `plane` at 82.4% duplication sits 4.4 times the floor and one block of `workshop` at 39.2% sits 1.1 times it. What the data support is a statement with levels, not a threshold:

- key duplication $\leq 45\%$ (21 blocks): below the floor, with one exception at $1.1\times$;
- 82.4% (1 file): $4.4\times$ the floor, and saturating in the window conditions (+6.86, +7.06, +7.01%);
- 100% (4 blocks): 300 to $500\times$ the floor, and graded in the window conditions.

A controlled sweep, and what it does not settle. The comparison above is observational: key duplication is confounded with everything else that distinguishes one survey from another. In a separate experiment we hold the input order at the identity and manipulate the key alone, by quantising the timestamp to a target duplication rate, with a dither condition that restores uniqueness at the same quantisation step as a control. Two files, in Table 5.

The 100% row is the clean result: manufacturing a fully degenerate key raises the cost of a permutation from nothing to +46.2 and +125.9%, and restoring uniqueness by dithering at the same quantisation step returns it to zero. The effect is caused by degeneracy of the key and not by the loss of timestamp resolution that produced it.

The intermediate rows do not agree with each other. On AHN4 the cost of a permutation tracks the reverse-order control to within a percentage point at 25, 50 and 75%: there is no order-specific excess until the key is fully degenerate. On vegetation there is an excess that grows (+3.0, +4.1, +8.0 percentage points over the control). One file looks abrupt and the other graded. We cannot resolve this, for a reason we can state: on AHN4 the dither control does not fire at the intermediate levels, because the quantisation step there is close to the floating-point spacing of the timestamps and there is no room to separate the groups again. The intermediate part of the sweep therefore has no working control on the file where the answer differs.

Note also that the controlled effect at 100% (+46.2, +125.9%) is far smaller than the +509.7 to +787.4% observed on the two files that happen to arrive with

degenerate keys. Those two are also unusual in other ways; the observational figure is not the effect of degeneracy alone.

The correlation does not carry weight here. Rank correlation between key duplication and the cost of a permutation is $+0.697$ ($p = 0.025$, $n = 10$ non-degenerate files) for the scan coder. There are nine such tests in the paper; the Bonferroni bound is $p \times 9 = 0.23$. Removing any one of four files from the ten destroys significance. We report it for completeness and do not build on it.

7 What this means for benchmarks

The order must be part of the test conditions. A lossless bitrate reported without an order condition is not determined. The worst case, a uniform permutation, costs a sequential coder a median of $+121.9\%$ (interquartile range $+89$ to $+140\%$, extremes $+36.0$ and $+828.6\%$ over the 26 blocks). The rearrangement that actually occurs in practice is milder and its sign is not even fixed: sorting an acquisition-order file into a space-filling-curve order, which is what the standard tools do, costs a median of $+16.1\%$ but ranges from -21.4% to $+304.1\%$ across our blocks. Common test conditions that fix the sequences, the quantisation and the attribute set, but not the arrangement, do not fix the number they are comparing — and the practical spread, with its sign change, is harder to reason about than the worst case.

Reordering the input is not a levelling operation. It is tempting to sort every input into a canonical order so that all coders face the same thing. That destroys the premise of the order-dependent coders instead of neutralising an advantage: the identity order is where an acquisition-order coder is designed to work, and Morton order is not neutral either (Section 5).

The comparison must be within a row of Table 1. If a multiset coder is compared with a sequence coder, the $\log_2(n!)/n = 18.489$ bits per point that only one of them carries have to be accounted for on one side or the other. We note that this figure applies to the 21 blocks of 10^6 points and not to the five smaller ones, and that it is the information content of the permutation, not the cost a coder pays for disorder; the two are not directly comparable.

We should also record a limit of the multiset column itself. G-PCC is exactly order-invariant here because every block fits in one slice. Above `sliceMaxPoints` the assignment of points to slices is made in input order, so a multiset coder is no longer strictly invariant either. Real files exceed that threshold, so an order clause cannot simply exempt the multiset side.

A minimum specification. An order clause for lossless point cloud test conditions needs to fix: (i) which arrangement is the baseline, (ii) whether coders may re-sort internally and whether the original order must be recoverable, (iii) how the permutation is accounted for when the two sides of a comparison differ on (ii), and (iv) a null control with a stated resolution, so

that a reported difference can be separated from what a benign rearrangement costs.

8 Limitations

Sample size. Twelve files, of which four airborne tiles share a footprint and two share a site: eight independent sites, six among the non-degenerate. A power calculation for $\rho = 0.7$ at 80% needs about 13. The correlational part of this study is underpowered and we do not rest any conclusion on it. The two endpoints of Section 6 do not need it.

Confounding. In the observational part, key duplication is confounded with everything else that distinguishes one survey from another. The controlled sweep breaks the confound only at full degeneracy: at intermediate levels the dither control does not fire on one of the two files, because the quantisation step is close to the floating-point spacing of the timestamps. The causal statement therefore holds at 100% and is open below it.

The gap in the key axis. In the observational data there is nothing between 45% and 82% duplication, and only one file above 45%. The shape of the boundary is open.

Variance. The three blocks of a large file give a within-file spread but not a between-file one; the five small files remain single blocks, and the correlation still has one point per file.

Accounting asymmetry. The scan coder’s figure covers X, Y, Z only, and those columns cannot be decoded without the timestamp and flight line that form the key. Adding the side columns raises the cost of a permutation to $+137.9\%$. That figure comes from a pilot run on a different block and, unlike the matrix, its per-candidate values were not round-trip verified; it is the only estimate we have and we mark it as weak. The correct statement is conditional: *given* the side columns, the geometry cost of the scan coder is order-independent at the key duplication rates where we can show it, which is 45% and below.

Scope. LAS-family files only, and airborne LiDAR for every large block. No automotive (KITTI-style) or terrestrial laser scanning data, and no sequence from the MPEG common test conditions. Two of the four coders are our own and unpublished; the headline figure for the other row, LASzip at $+121.9\%$, does not depend on them.

Novelty. Every component of the invariance property is prior art (Section 2). What is new here is the measurement and the protocol, together with the finding that the property is governed by degeneracy of the key and the quantification of what happens when the key degenerates.

9 Conclusion

Input order is a factor with a median effect of $+121.9\%$ (LASzip) and $+130.9\%$ (our sequential coder) under a uniform permutation, it is absent from the test conditions used to compare lossless point cloud coders, and

it interacts with coder internals in ways that are not even monotone in sign. Making it explicit costs one extra axis in the experimental design and two null controls. Leaving it implicit makes the reported numbers irreproducible.

One practical note. Between the first and second version of this experiment we replaced the larger null control with the smaller one, and we stated a condition on the sort key that our own data contradict. Both changes made the result look cleaner, and neither was caught by re-reading; both were caught by checking the tables against the raw measurements. A protocol for order sensitivity should require the raw per-block values to be published alongside any summary, for the same reason.

References

- [1] M. Isenburg, “LASzip: lossless compression of LiDAR data,” *Photogrammetric Engineering and Remote Sensing*, 79(2), 2013.
- [2] J. Kunze et al., “Entropy coding of unordered data structures,” *ICLR*, 2024.
- [3] D. Severo et al., “Compressing multisets with large alphabets,” *IEEE Journal on Selected Areas in Information Theory*, 2022.
- [4] Lossless coding of airborne LiDAR attributes in a geometry-derived order. Companion manuscript.
- [5] SerLiC: serialised coding of LiDAR point clouds. *ICML*, 2025.